

گزارش پایانی پروژه

موضوع

طراحی و ارزیابی رویکردی برای بهبود زمان‌بندی وظایف یادگیری ماشین در Kubernetes

سامانه پیاده‌سازی شده: KubeML-Aware

امیر میرزائی موحد

مجید بافنده

شماره دانشجویی: ۴۰۴۱۲۵۰۵۹

شماره دانشجویی: ۴۰۴۱۳۱۰۴۳

۱. خلاصه پروژه

در این پروژه یک لایه پندیرش آگاه از رتبه برای بارهای کاری آموزش یادگیری ماشین در Kubernetes با نام KubeML-Aware طراحی و پیاده‌سازی شد. برخلاف رویکردهایی که زمان‌بند پیش‌فرض را جایگزین می‌کنند، این سامانه تنها زمان و اجده‌های پادها برای زمان‌بندی را کنترل می‌کند و وظایف فیلترکردن گره‌ها، امتیازدهی و اتصال (binding) را همچنان به زمان‌بند پیش‌فرض واگذار می‌کند. سازوکار کار به این صورت است که همه پادهای یک دسته (burst) پشت یک دروازه زمان‌بندی پایدار (Pod Scheduling Gate) نگه داشته می‌شوند؛ پس از کامل شدن دسته، یک کنترلر شش ویژگی هر وظیفه را اعتبارسنجی می‌کند، یک رتبه قطعی (deterministic) محاسبه می‌کند و دروازه‌ها را به ترتیب رتبه و بر اساس بازخورد منابع برمی‌دارد. این طراحی امکان بهبود ترتیب اجرای وظایف را بدون از دست دادن ایمنی و پایداری زمان‌بند اصلی فراهم می‌کند.

۲. دستاوردهای اصلی

مهم‌ترین خروجی‌های عملی پروژه عبارت‌اند از: پیاده‌سازی کنترلر به صورت یک سرویس درون‌خوشه‌ای (in-cluster) به جای شبیه‌ساز برون‌خط؛ الگوریتم رتبه‌بندی شش ویژگی با نرمال‌سازی درون‌دسته‌ای و مدیریت هم‌رتبگی؛ سازوکار FastPath که با استفاده از تلمتری تازه منابع، فاصله‌های خالی صف زمان‌بند را حذف می‌کند و در صورت کهنه‌بودن داده‌ها به صورت ایمن (fail-closed) عمل می‌کند؛ یک بهینه‌ساز دُنباله (tail-balancer) برای بازچینش وظایف کم‌اهمیت‌تر؛ بسته استقرار Helm به همراه تنظیمات RBAC؛ و در نهایت اجرای یک آزمایش واقعی روی خوشه K3s. در ارزیابی، میانگین زمان تکمیل وظیفه (JCT) از حدود ۳۱/۰۵ ثانیه به ۲۸/۷۶ ثانیه کاهش یافت که به طور میانگین معادل بهبود جفتی ۷/۳۱ درصد است. علاوه بر این، مقاله‌ای علمی به سبک مجلات به همراه شکل‌ها، جداول و تحلیل آماری برای مستندسازی این کار تدوین شد.

۳. فعالیت‌های انجام شده

با توجه به دو نفره بودن پروژه، حجم کار به گونه‌ای انتخاب شد که فراتر از یک پروژه تک‌نفره باشد؛ علاوه بر طراحی الگوریتم و نگارش مقاله، یک سامانه کامل و آماده استقرار در Kubernetes همراه با مجموعه گسترده‌ای از آزمون‌ها پیاده‌سازی و روی یک خوشه واقعی ارزیابی شد. فعالیت‌های پروژه را می‌توان در چهار حوزه اصلی زیر دسته‌بندی کرد:

الف) پیاده‌سازی و توسعه سامانه. در این بخش، هسته کنترلر دروازه‌ای (gate controller)، الگوریتم رتبه‌بندی (rank)، منطق اجرا و ثبت رویدادها (records, execution)، سازوکار جمع‌آوری دسته (burst)، مسیر سریع (fast_path)، تنظیم آهنگ تطبیقی (pacing) و مدل کار مصنوعی آموزش (work_model) توسعه یافتند. همچنین بسته Helm، منابع RBAC، سرویس استنتاج (inference) و مانیفست‌های Knative برای استقرار امن آماده شدند.

ب) آزمایش و جمع‌آوری داده. مجموعه‌ای از بارهای کاری متنوع (کوچک، متوسط، بزرگ، ناهمگون و ابلیشن) طراحی شد و پروتکل آزمایش جفتی (paired) روی خوشه K3s با پنج تکرار و دوازده وظیفه در هر بازو اجرا گردید. کد جمع‌آوری متریک‌ها (metrics_collector)، تلمتری و ثبت رویدادهای سطح برنامه برای اندازه‌گیری دقیق JCT و

makespan توسعه داده شد.

ج) تحلیل نتایج و نگارش مقاله. داده‌های خام آزمایش با اسکریپت‌های تحلیل آماری (analyze, statistics) پردازش شد و بازه‌های اطمینان جفتی محاسبه گردید. سپس مقاله نهایی به سبک IEEE به همراه شکل‌های معماری، بافت Kubernetes، نمودار نتایج و خط‌زمانی اجرا تدوین و در چند مرحله بازبینی و مقاوم‌سازی (reviewer-hardening) شد.

د) آزمون و رفع اشکال. یک مجموعه جامع از آزمون‌های واحد و یکپارچگی برای همه بخش‌ها (رتبه‌بندی، کنترلرها، اجرا، مسیر سریع، تلمتری، تولید بار کاری و اعتبارسنجی مانیفست‌ها) نوشته شد. در این مرحله باگ‌های مربوط به تجزیه لاگ‌ها و مدیریت quoteها شناسایی و برطرف شد و صحت مانیفست‌های Kubernetes پیش از استقرار اعتبارسنجی گردید.

۴. نقش و سهم هر یک از اعضا

مطابق خواسته درس، نقش هر عضو در ادامه به صورت دقیق و صریح مشخص شده است. نکته اصلی این است که هر دو عضو در تمام بخش‌های پروژه — شامل نگارش مقاله، کدنویسی، اجرای آزمایش‌ها و رفع اشکال — به صورت فعال مشارکت داشته‌اند و هیچ بخشی به طور کامل بر عهده تنها یک نفر نبوده است. تصمیم‌های اصلی طراحی، ساختار مقاله و تحلیل نتایج به صورت مشترک و در جلسات کاری مشترک اتخاذ شده است. با این حال، برای شفافیت بیشتر، مسئولیت اصلی/راهبری هر حوزه به شکل زیر تقسیم شده است:

حوزه کاری	مسئول اصلی / راهبر	مشارکت مشترک
طراحی و پیاده‌سازی کنترلر و الگوریتم رتبه‌بندی	امیر میرزائی موحد	هر دو نفر
اجرای آزمایش‌ها و جمع‌آوری داده روی خوشه	امیر میرزائی موحد	هر دو نفر
تحلیل آماری نتایج و نگارش/بازبینی مقاله	مجید بافنده	هر دو نفر
طراحی بارهای کاری، آزمون‌ها و رفع اشکال	مجید بافنده	هر دو نفر

امیر میرزائی موحد به عنوان راهبر بخش پیاده‌سازی، تمرکز بیشتری بر توسعه هسته کنترلر دروازه‌ای، الگوریتم رتبه‌بندی شش‌ویژگی، سازوکار FastPath و برپاسازی و اجرای آزمایش‌ها روی خوشه K3s داشت. در کنار این، او در نگارش بخش‌های فنی مقاله، بازبینی تحلیل‌ها و نوشتن بخشی از آزمون‌ها نیز مشارکت مستقیم داشت.

مجید بافنده به عنوان راهبر بخش ارزیابی و مستندسازی، تمرکز بیشتری بر طراحی بارهای کاری متنوع، تحلیل آماری داده‌ها، محاسبه بازه‌های اطمینان و نگارش و مقاوم‌سازی مقاله داشت. او همچنین در توسعه بخش‌هایی از کد کنترلر، نوشتن آزمون‌ها و فرایند رفع اشکال به صورت فعال همکاری کرد.

به طور خلاصه، تقسیم‌بندی بالا صرفاً بیانگر تمرکز اصلی هر فرد است و نه مرزبندی قطعی؛ هر دو عضو در کل چرخه پروژه از ایده‌پردازی تا استقرار و نگارش گزارش، سهم برابر و متوازی داشته‌اند.

۵. نتایج کلیدی

ارزیابی روی یک گره چهارهسته‌ای اشتراکی K3s و با پنج تکرار جفتی انجام شد. نتایج اصلی در جدول زیر خلاصه شده‌اند. میانگین زمان تکمیل وظیفه در تمام پنج جفت بهبود یافت و بهبود میانگین جفتی برابر ۷/۳۱ درصد (با بازه اطمینان اکتشافی ۹۵ درصد برابر [۲/۵۷، ۱۲/۰۵]) به دست آمد. زمان کل اتمام دسته (makespan) نیز در هر

پنج جفت در همان جهت کاهش یافت (۲/۴۷ درصد)، اما چون بازه اطمینان آن شامل صفر است، به عنوان یک روند جهت دار و نه یک اثر قطعی گزارش می شود.

معیار	FIFO پایه	KubeML-Aware	بهبود جفتی
میانگین JCT	۳۱/۰۵ ثانیه	۲۸/۷۶ ثانیه	+۷/۳۱٪
Makespan	۵۵/۳۷ ثانیه	۵۴/۰۱ ثانیه	+۲/۴۷٪

۶. چالش ها و جمع بندی

مهم ترین چالش پروژه، اجرای آزمایش روی یک گره عملیاتی اشتراکی (نه یک میزبان محک گیری ایزوله) بود که برای مهار آن از پروتکل جفتی، تناوب ترتیب بازوها، بررسی خنک بودن گره و ورودی های یکسان استفاده شد. چالش دیگر، حفظ ایمنی زمان بند پیش فرض هم زمان با اعمال سیاست رتبه بندی بود که با استفاده از دروازه های زمان بندی پایدار و طراحی fail-closed حل شد. در مجموع، این پروژه نشان داد که می توان یک سیاست پذیرش آگاه از ویژگی های یادگیری ماشین را بدون جایگزینی موتور مکان یابی Kubernetes افزود و به بهبود قابل مشاهده در میانگین زمان تکمیل وظایف رسید. نتایج به دست آمده امیدوارکننده ولی مقدماتی اند و گام بعدی، اجرای آزمایش روی خوشه اختصاصی با تکرار بیشتر و بارهای کاری واقعی چارچوب های یادگیری ماشین است.

منابع

- [۱] M. Abdelshaheed and M. Ashour, "Intelligent scheduling of AI tasks in Kubernetes," in *Proceedings of the ۳۵th International Conference on Computer Theory and Applications (ICCTA)*, ۲۰۲۵ pp. —۲۴.۳۱ DOI: 10.1109/ICCTA68914.2025.11519965
- [۲] Q. Weng et al., "MLaaS in the wild: Workload analysis and scheduling in large-scale heterogeneous GPU clusters," in *۱۹th USENIX Symposium on Networked Systems Design and Implementation (NSDI)* (۲۲, USENIX Association, ۲۰۲۲ pp. —۹۴۵.۹۶۰
- [۳] M. Alhakimi and R. Latip, "Enhancing the Kubernetes scheduler: A state-of-the-art review from cloud to edge," *Computers*, vol. ۱۵ no. ۷ p. ۴۵۸.۲۰۲۶ DOI: 10.3390/computers15070458
- [۴] Kubernetes Authors. *Scheduling framework*, Kubernetes Documentation. Accessed: Aug. ۱۰, ۲۰۲۶.۲۰۲۶ [Online]. Available: <https://kubernetes.io/docs/concepts/scheduling-eviction/scheduling-framework/>
- [۵] Kubernetes Authors. *Pod scheduling readiness*, Kubernetes Documentation. Accessed: Aug. ۱۰, ۲۰۲۶.۲۰۲۶ [Online]. Available: <https://kubernetes.io/docs/concepts/scheduling-eviction/pod-scheduling-readiness/>
- [۶] Kubernetes Authors. *Resource metrics pipeline*, Kubernetes Documentation. Accessed: Aug. ۱۰, ۲۰۲۶.۲۰۲۶ [Online]. Available: <https://kubernetes.io/docs/tasks/debug/debug-cluster/resource-metrics-pipeline/>
- [۷] Kubernetes Authors. *Workload API*, Kubernetes Documentation. Accessed: Aug. ۱۰, ۲۰۲۶.۲۰۲۶ [Online]. Available: <https://kubernetes.io/docs/concepts/workloads/workload-api/>
- [۸] J. Gu et al., "Tiresias: A GPU cluster manager for distributed deep learning," in *۱۶th USENIX Symposium on Networked Systems Design and Implementation (NSDI)* (۱۹, USENIX Association, ۲۰۱۹ pp. —۴۸۵.۵۰۰

- [۹] W. Xiao et al., “Gandiva: Introspective cluster scheduling for deep learning,” in *13th USENIX Symposium on Operating Systems Design and Implementation (OSDI)* (۱۸, USENIX Association, ۲۰۱۸ pp. —۵۹۵.۶۱۰
- [۱۰] S. J. Subramanya, D. Arfeen, S. Lin, A. Qiao, Z. Jia, and G. R. Ganger, “Sia: Heterogeneity-aware, goodput-optimized ML-cluster scheduling,” in *Proceedings of the 29th Symposium on Operating Systems Principles*, ۲۰۲۳ pp. —۶۴۲.۶۵۷ DOI: 10.1145/3600006.3613175
- [۱۱] P. Zheng, R. Pan, T. Khan, S. Venkataraman, and A. Akella, “Shockwave: Fair and efficient cluster scheduling for dynamic adaptation in machine learning,” in *20th USENIX Symposium on Networked Systems Design and Implementation (NSDI)* (۲۳, USENIX Association, ۲۰۲۳ pp. —۷۰۳.۷۲۳
- [۱۲] S. Rajasekaran, M. Ghobadi, and A. Akella, “CASSINI: Network-aware job scheduling in machine learning clusters,” in *21st USENIX Symposium on Networked Systems Design and Implementation (NSDI)* (۲۴, USENIX Association, ۲۰۲۴ pp. —۱۴۰۳.۱۴۲۰
- [۱۳] X. Zhang et al., “Rubick: Exploiting job reconfigurability for deep learning cluster scheduling,” in *Proceedings of Machine Learning and Systems*, vol. ۷, ۲۰۲۵
- [۱۴] S. Dong, B. Zheng, L. Pan, and S. Liu, “A reinforcement learning-based approach for scheduling ML training tasks in heterogeneous Kubernetes clusters,” *Future Generation Computer Systems*, vol. ۱۸۲ p. ۱۰۸,۴۵۹.۲۰۲۶ DOI: 10.1016/j.future.2026.108459
- [۱۵] J. Santos et al., “Sakkara: Intelligent topology-aware scheduling for Kubernetes in the age of AI,” *IEEE Transactions on Network and Service Management*, vol. ۲۳ pp. —۵۴۳۷, ۵۴۵۱.۲۰۲۶ DOI: 10.1109/TNSM.2026.3703831
- [۱۶] Kubernetes SIG Scheduling, *Kueue overview*, Kueue Documentation, Accessed: Aug. ۱۰, ۲۰۲۶.۲۰۲۶ [Online]. Available: <https://kueue.sigs.k8s.io/docs/overview/>
- [۱۷] Volcano Community, *Unified scheduling*, Volcano Documentation, Accessed: Aug. ۱۰, ۲۰۲۶.۲۰۲۶ [Online]. Available: <https://volcano.sh/docs/keyfeatures/unifiedscheduling/>
- [۱۸] NVIDIA, *KAI scheduler*, GitHub repository, Accessed: Aug. ۱۰, ۲۰۲۶.۲۰۲۶ [Online]. Available: <https://github.com/kai-scheduler/KAI-Scheduler>
- [۱۹] C. R. Harris, K. J. Millman, S. J. van der Walt, et al., “Array programming with NumPy,” *Nature*, vol. ۵۸۵ pp. —۳۵۷.۳۶۲.۲۰۲۰ DOI: 10.1038/s41586-020-2649-2
- [۲۰] K3s Authors, *K3s: Lightweight Kubernetes*, K3s Documentation, Accessed: Aug. ۱۰, ۲۰۲۶.۲۰۲۶ [Online]. Available: <https://docs.k3s.io/>
- [۲۱] A. M. Movahed, *KubeML-Aware: Source code and experiment artifacts*, GitHub repository, Accessed: Aug. ۱۰, ۲۰۲۶.۲۰۲۶ [Online]. Available: <https://github.com/AmirMirzaeiMovahed/KubeML-Aware>